Newton RL Robot Control Pipeline

Detailed Technical Reference

Based on: example_robot_policy.py & anymal_robot_policy.py

Newton Physics Engine v1.0.0 (MuJoCo / Warp backend)

1. System Overview

The Newton RL robot control system enables real-time control of articulated robots using pretrained reinforcement learning (RL) policies. The system implements a classic sim-to-sim transfer pipeline: policies originally trained in NVIDIA Isaac Lab are deployed inside the Newton physics simulator.

The pipeline follows a hierarchical control architecture:

1. A human operator provides high-level velocity commands via keyboard
2. An RL neural network policy converts commands into joint-level actions
3. A PD servo controller converts actions into joint torques
4. The physics engine integrates the dynamics

1.1 Supported Robots

Robot	Asset Directory	DOFs	Policy Source	Physics
ANYmal C	anybotics_anymal_c	12	MJW + PhysX	Quadruped
Unitree Go2	unitree_go2	12	MJW + PhysX	Quadruped
Unitree G1 29	unitree_g1	29	MJW only	Humanoid
Unitree G1 23	unitree_g1	23	MJW + PhysX	Humanoid

1.2 File Structure

- example_robot_policy.py: Multi-robot example supporting all 4 robot configurations
- any

2. Complete Control Pipeline

2.1 Stage 1: Keyboard Input to Command Vector

The operator controls the robot via 6 keyboard keys that map to a 3D velocity command vector. This command represents the desired base velocity in the robot's local frame. The command is NOT a joint-level reference -- it is a high-level locomotion directive.

Key Mapping

Key	Action	Command Index	Value
i	Move forward	command[0]	+1.0 m/s
k	Move backward	command[0]	-1.0 m/s
j	Strafe left	command[1]	+0.5 m/s
l	Strafe right	command[1]	-0.5 m/s
u	Turn left	command[2]	+1.0 rad/s
o	Turn right	command[2]	-1.0 rad/s
p	Reset robot	N/A	Full state reset

When no key is pressed, the command is (0, 0, 0), meaning "stand still". The command persists for one policy step, then resets to zero. Lateral velocity is capped at 0.5 m/s (vs 1.0 for forward/yaw) because sideways locomotion is typically less stable.

Implementation (example_robot_policy.py, lines 330-355)

```
def key_handler(self):
    command = torch.zeros(3, device=self.device)
    if keys['i']: command[0] = 1.0   # forward
    if keys['k']: command[0] = -1.0  # backward
    if keys['j']: command[1] = 0.5   # left
    if keys['l']: command[1] = -0.5  # right
    if keys['u']: command[2] = 1.0   # turn left
    if keys['o']: command[2] = -1.0  # turn right
    return command
```

2.2 Stage 2: Observation Construction

The command vector is combined with the robot's proprioceptive state to form the observation vector that the RL policy network expects as input. This is computed in the compute_obs() function. All quantities are transformed into the robot's body frame using quaternion rotation, since the policy was trained with body-frame observations.

Observation Vector Layout (48 dimensions for ANYmal)

Indices	Component	Description	Details
0:3	Base lin. velocity	Linear velocity in body frame	v_body = quat_rotate_inv(q, v_world)
3:6	Base ang. velocity	Angular velocity in body frame	omega_body = quat_rotate_inv(q, omega)
6:9	Projected gravity	Gravity vector in body frame	g_body = quat_rotate_inv(q, [0,0,-1])
9:12	Command	Velocity command from keyboard	[v_x, v_y, yaw_rate]

12:24	Joint pos. offset	Joint angles minus defaults	q_current - q_initial
24:36	Joint velocities	Joint angular velocities	dq (rad/s)
36:48	Previous actions	Actions from last timestep	Temporal continuity signal

Quaternion Rotation (quat_rotate_inverse)

All world-frame vectors are converted to the body frame using the inverse quaternion rotation. This is implemented as a TorchScript-compiled function for performance:

```
@torch.jit.script
def quat_rotate_inverse(q, v):
    q_w = q[:, 0]    # scalar part
    q_vec = q[:, 1:]  # vector part (x, y, z)
    a = v * (2.0 * q_w**2 - 1.0)
    b = torch.cross(q_vec, v) * q_w * 2.0
    c = q_vec * (q_vec * v).sum(-1, keepdim=True) * 2.0
    return a - b + c
```

Note: Newton uses (w, x, y, z) quaternion convention, matching MuJoCo. The function operates on batched tensors (shape [N, 4] for quaternions, [N, 3] for vectors).

Projected Gravity

The projected gravity vector tells the policy which way is "down" relative to the robot's body. When the robot is upright, this is approximately [0, 0, -1]. When tilted, this rotates accordingly. This is critical for balance - without it, the policy wouldn't know the robot's orientation relative to gravity.

2.3 Stage 3: Neural Network Policy Inference

The observation vector is fed through a pretrained neural network that outputs one action value per actuated joint. The network was trained via Proximal Policy Optimization (PPO) in NVIDIA Isaac Lab.

Policy Network Architecture

The policies are stored as PyTorch JIT-compiled models (.pt files). They are typically Multi-Layer Perceptrons (MLPs) with the following typical structure:

- Input: 48-dimensional observation vector (for ANYmal; varies by robot)
- Hid

Policy Loading

```
self.policy = torch.jit.load(policy_path)
self.policy.to(self.device)
self.policy.eval()  # inference mode, no gradients
```

The policy runs in eval mode with no gradient computation for maximum performance. Inference takes <1ms on GPU, allowing real-time control at 50 Hz.

MJWarp vs PhysX Policies

Two types of pretrained policies are available:

MJWarp (mjw): Trained with MuJoCo-Warp physics backend in Isaac Lab. These transfer most naturally to Newton since Newton uses MuJoCo-based dynamics.

PhysX: Trained with NVIDIA PhysX backend. These may use a different joint ordering than Newton/MuJoCo. The code handles this with a joint index remapping:

```
# Find mapping between PhysX and MJWarp joint orders
indices = find_physx_mjwarp_mapping(
    mjwarp_joint_names,  # from Newton model
    physx_joint_names    # from YAML config
)
# Rearrange policy output to match Newton joint order
rearranged = action[:, indices]
```

2.4 Stage 4: Action to Joint Position Target

The raw action from the policy is converted into absolute joint position targets via a simple affine transformation:

```
target_positions = joint_pos_initial + action_scale * actions
```

Parameters

Parameter	Typical Value	Description
joint_pos_initial	Standing pose (rad)	Default joint angles from YAML config; the nominal pose
action_scale	0.5 (ANYmal)	Scaling factor from YAML; limits max deviation from default
actions	[-1, 1] (approx)	Raw policy output; dimensionless

This means the policy can command joint positions within +/- action_scale radians of the default standing pose. For ANYmal with action_scale=0.5, each joint can deviate at most ~28.6 degrees from its nominal position per step. This keeps the robot in a safe operating region.

Floating Base Handling

Newton models the robot as a floating-base system. The first 6 degrees of freedom (DOFs) represent the base position (3 translational) and orientation (3 rotational). These are unactuated -- the robot cannot directly control its base pose. Only gravity and ground contact forces affect the base.

```
# Prepend 6 zeros for the unactuated floating base
a_with_zeros = torch.cat([
    torch.zeros(6, device=device),  # base: no actuation
    target_positions.squeeze(0)      # joint targets
])

# Write to Newton control structure
a_wp = wp.from_torch(a_with_zeros)
```

```
wp.copy(control.joint_target_pos, a_wp)
```

2.5 Stage 5: PD Servo Controller (model.control())

This is where Newton's physics engine takes over. The control.joint_target_pos values are used by an internal PD (Proportional-Derivative) controller to compute joint torques. This happens inside the solver's step() function.

PD Control Law

For each actuated joint i, the applied torque is computed as:

```
tau_i = Kp * (q_target - q_current) - Kd * dq_current
```

Where:

- tau_i: Applied joint torque (N*m)
- Kp

PD Gains by Robot

Robot	Kp (stiffness)	Kd (damping)	Notes
ANYmal C	300 N*m/rad	10 N*m*s/rad	High stiffness for 50kg robot
Unitree Go2	100 N*m/rad	5 N*m*s/rad	Lighter robot, lower gains
G1 29DOF	Varies/joint	Varies/joint	Different gains for arms/legs
G1 23DOF	Varies/joint	Varies/joint	Reduced DOF variant

JointTargetMode.POSITION

CRITICAL: Each actuated DOF must be explicitly configured for position control mode during model building. Without this, the joints default to passive (no actuation) and the robot collapses under gravity as a ragdoll. This is set during model construction:

```
from newton import JointTargetMode

for i in range(num_dofs):
    builder.joint_target_ke[i + 6] = stiffness  # Kp gain
    builder.joint_target_kd[i + 6] = damping     # Kd gain
    builder.joint_target_mode[i + 6] = int(JointTargetMode.POSITION)
    # +6 offset skips the unactuated floating base DOFs
```

The JointTargetMode enum has the following values:
NONE (0): Passive joint, no actuation
POSITION (1): PD position control (used here)
VELOCITY (2): PD velocity control
FORCE (3): Direct torque control

2.6 Stage 6: Physics Simulation (solver.step())

The computed torques are applied to the articulated rigid body system, and the physics engine integrates the equations of motion forward in time.

Simulation Parameters

Parameter	Value	Description
sim_dt	0.005 s	Physics timestep (200 Hz integration)
decimation	4	Physics steps per policy step
policy_dt	0.02 s	Effective policy period (50 Hz)
sim_substeps	1	Substeps per physics step (for contact)
num_envs	1	Number of parallel environments

Simulation Loop Structure

```
def simulate(self):
    # Run policy at 50 Hz (every 0.02s of sim time)
    key_handler()       # Read keyboard
    obs = compute_obs() # Build observation
    act = policy(obs)   # Neural network inference
    set_targets(act)    # Write to control.joint_target_pos

    # Integrate physics at 200 Hz (4 substeps)
    for i in range(decimation):  # 4 iterations
        state_1 = solver.step(
            model, state_0, control, contacts=None, dt=sim_dt
        )
        state_0.assign(state_1)  # swap states

    # Update contact information for rendering
    solver.update_contacts(contacts, state_0)
```

The decimation approach means the neural network runs at 50 Hz while physics runs at 200 Hz. This balances policy computational cost against physics accuracy. The same torques are applied for all 4 substeps within one policy step.

State Management

Newton uses a double-buffering scheme for simulation states:

state_0: Current state (input to solver)
state_1: Next state (output of solver)

After each substep, state_0.assign(state_1) copies the result back. The State object contains:

- body_q: Body positions and orientations (quaternions)
- bod

3. YAML Configuration File

Each robot has a YAML configuration file (e.g., anymal.yaml) that defines all parameters needed to reproduce the training environment. This ensures the deployed policy sees the same dynamics as during training.

3.1 Complete Configuration Fields

Field	Example Value	Description
action_scale	0.5	Multiplier on raw policy output
stiffness	300.0	PD proportional gain Kp (N*m/rad)
damping	10.0	PD derivative gain Kd (N*m*s/rad)
sim_dt	0.005	Physics timestep (seconds)
decimation	4	Physics steps per policy inference
joint_pos_initial	[list of floats]	Default standing joint angles (rad)
joint_names	[list of strings]	Joint names for PhysX ordering
sim_substeps	1	Contact solver iterations per step

3.2 ANYmal C Default Joint Positions

The default standing pose defines the nominal joint configuration. The policy outputs offsets from this pose. For ANYmal C (12 DOFs, 3 per leg):

Joint	Default (rad)	Approx (deg)
LF_HAA (Left-Front Hip Ab/Ad)	0.03	1.7
LF_HFE (Left-Front Hip Flex/Ext)	0.4	22.9
LF_KFE (Left-Front Knee)	-0.8	-45.8
RF_HAA	-0.03	-1.7
RF_HFE	0.4	22.9
RF_KFE	-0.8	-45.8
LH_HAA	0.03	1.7
LH_HFE	-0.4	-22.9
LH_KFE	0.8	45.8
RH_HAA	-0.03	-1.7
RH_HFE	-0.4	-22.9
RH_KFE	0.8	45.8

4. Model Building and Initialization

4.1 Asset Loading

Robot assets are loaded from the newton-assets repository. The download_asset() function fetches and caches the assets locally:

```
asset_dir = newton.utils.download_asset(config.asset_dir)
asset_path = os.path.join(asset_dir, config.asset_path)
yaml_path = os.path.join(asset_dir, config.yaml_path)
policy_path = os.path.join(asset_dir, config.policy_path[engine])
```

4.2 Model Construction Pipeline

The Newton model is built using a Builder pattern (newton.ModelBuilder):

```
# 1. Create builder
builder = newton.ModelBuilder()

# 2. Parse USD/URDF asset into builder
newton.load_asset(asset_path, builder)

# 3. Configure actuators for each DOF
for i in range(num_dofs):
    builder.joint_target_ke[i+6] = stiffness   # PD Kp
    builder.joint_target_kd[i+6] = damping      # PD Kd
    builder.joint_target_mode[i+6] = int(
        JointTargetMode.POSITION                 # Enable actuation!
    )

# 4. Build the model
model = builder.finalize()

# 5. Create solver (MuJoCo-based)
solver = newton.SolverMuJoCo(model)

# 6. Initialize states and controls
state_0 = model.state()
state_1 = model.state()
control = model.control()
```

4.3 External URDF Loading (anymal_robot_policy.py)

The anymal_robot_policy.py variant supports loading robot models from external URDF files instead of the bundled USD assets. This is useful for testing custom robot descriptions:

```
if args.external_model:
    # Load from local URDF file
    builder.add_urdf(
        urdf_path,
        mesh_dir=mesh_dir,
        floating=True,         # free-floating base
        density=1000.0,
```

```
        ensure_nonstatic=True
    )
else:
    # Load from newton-assets USD
    newton.load_asset(asset_path, builder)
```

5. Joint Ordering and Remapping

A critical detail in sim-to-sim transfer is that different physics engines may use different joint orderings. MuJoCo/Newton typically orders joints based on the kinematic tree traversal, while PhysX may use a different convention.

5.1 The Problem

If a policy was trained in PhysX, its output actions are ordered according to PhysX joint ordering. When deployed in Newton (MuJoCo), the joints may be in a different order. Applying actions in the wrong order means the left leg receives commands intended for the right leg, etc.

5.2 The Solution: find_physx_mjwarp_mapping()

```
def find_physx_mjwarp_mapping(mjwarp_names, physx_names):
    """Find index mapping from PhysX to MJWarp joint order.

    Returns indices such that:
      mjwarp_action[i] = physx_action[indices[i]]
    """
    indices = []
    for mj_name in mjwarp_names:
        for px_idx, px_name in enumerate(physx_names):
            if mj_name in px_name or px_name in mj_name:
                indices.append(px_idx)
                break
    return torch.tensor(indices)
```

For MJWarp-trained policies (the default), no remapping is needed -- the policy output ordering matches Newton's joint ordering directly.

6. Timing and Real-Time Control

6.1 Timing Budget

Component	Duration	Frequency
Policy inference	< 1 ms	50 Hz
Physics step (x4)	~2-4 ms	200 Hz (4 x 50 Hz)
Observation compute	< 0.5 ms	50 Hz

Rendering	~5-10 ms	50 Hz (with vsync)
Total per frame	~8-16 ms	Well within 20 ms budget

6.2 Real-Time Rate Limiting

The simulation loop includes a rate limiter to ensure physics runs at real-time speed, regardless of monitor refresh rate:

```
import time

self.policy_dt = frame_dt * decimation  # 0.02s
self.last_step_time = time.perf_counter()

# In step():
elapsed = time.perf_counter() - self.last_step_time
sleep_time = self.policy_dt - elapsed
if sleep_time > 0:
    time.sleep(sleep_time)
self.last_step_time = time.perf_counter()
```

Without this limiter, on a 144 Hz monitor the simulation would run at ~2.9x real-time, and on a 240 Hz monitor at ~4.8x. This causes the robot to appear accelerated and makes keyboard control difficult.

7. State Reset (Key 'p')

Pressing 'p' resets the robot to its initial configuration. This is implemented by restoring the initial state back to state_0:

```
def reset(self):
    self.state_0 = self.initial_state.clone()
    self.actions = torch.zeros_like(self.actions)
    self.command = torch.zeros(3, device=self.device)
    # Robot returns to default standing pose
```

This is essential for recovery when the robot falls or reaches an unrecoverable state. The initial state corresponds to the robot standing at the default joint configuration with zero velocity.

8. Complete Control Flow Summary

The following shows the complete data flow from keyboard to physics:

```
KEYBOARD INPUT                          PHYSICS OUTPUT
  i/k/j/l/u/o                             Joint torques
      |                                        |
      v                                        v
 [3D command]                            [PD Controller]
  vx, vy, wz                           tau = Kp*(q_t - q)
      |                                   - Kd * dq
      v                                      ^
 [Observation]                               |
```

```
 48-dim vector  ------>  [RL Policy]  ->  [action]
 (body frame)           Neural Net       [-1, 1]
     ^                                       |
     |                                       v
[State readout]                          [Position target]
 joint_q, joint_qd                    q_target = q_init +
 body_q, body_qd                      action_scale * act
     ^                                       |
     |                                       v
[solver.step()]  <----  [control.joint_target_pos]
 200 Hz physics           Written via wp.copy()
 (4 substeps)

Full loop runs at 50 Hz (0.02s per policy step)
```

9. Common Issues and Troubleshooting

Symptom	Cause	Fix
Robot collapses on ground	JointTargetMode not set	Set builder.joint_target_mode to POSITION for each DOF
Robot accelerated/fast	No frame rate limiter	Add time.sleep() rate limiting to step()
Robot walks without input	Policy drift (sim-to-sim)	Ensure correct timing; inherent in some policies
Limbs move wrong direction	Joint order mismatch	Use find_physx_mjwarp_mapping() for PhysX policies
ImportError: opengl_module	Removed in Newton 1.0	Remove ENABLE_CUDA_INTEROP lines
model.collide() not found	API changed in Newton 1.0	Use newton.Contacts() + solver.update_contacts()
test() not called	Renamed in Newton 1.0	Rename to test_final()

10. Newton API Migration (Pre-1.0 to 1.0.0)

Summary of all breaking API changes that affect the robot policy examples:

Old API	New API (1.0.0)	Notes
(implicit joint mode)	builder.joint_target_mode[i] = POSITION	Must be explicit
model.collide(state)	newton.Contacts(max_count, 0)	Contacts created separately
model.collide() in substep	solver.step(..., contacts=None)	Contacts handled internally
Manual __dict__ state swap	state_0.assign(state_1)	Direct method call
N/A	solver.update_contacts(c, s)	New method after substep loop
opengl.ENABLE_CUDA_INTEROP	(removed)	No longer needed
def test(self)	def test_final(self)	Method renamed